Entwicklung des Computerspiels "Mau-Mau" in der Programmiersprache Java

Assignment im Modul JAV41

von

[redacted]

Im Studiengang Software Engineering
an der staatlich anerkannten AKAD Hochschule Stuttgart

4. Juli 2023

Bearbeitungszeit	8 Wochen
Betreuerin	Prof.in Dr. Andrea Herrmann
Immatrikulationsnummer	[redacted]
E-Mail	[redacted]
Adresse	

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung und Ziel der Arbeit 1
1.2 Aufbau der Arbeit 1

2 Grundlagen **2**
2.1 Phasen der Softwareentwicklung 2
2.2 Die objektorientierte Programmiersprache Java 2
2.3 UML (Unified Modeling Language) 3

3 Softwareentwicklung eines Spiels mittels Java **4**
3.1 Anforderungen an das Spiel und die Spielregeln 4
3.2 Klassendiagramm als Entwurf 5
3.3 Implementierung der Spiellogik 6
3.3.1 Klassen und Vererbung 6
3.3.2 Assoziationen 7
3.3.3 Karten und Kartenstapel 7
3.3.4 Mischen-Algorithmus 8
3.3.5 Spielzug 8
3.4 Implementierung der GUI (Graphical User Interface) 10
3.5 Phasen nach der Implementierung 12
3.5.1 Testphase 12
3.5.2 Inbetriebnahme, Wartung und Weiterentwicklung 12

4 Fazit **13**
4.1 Zusammenfassung 13
4.2 Kritische Würdigung und Ausblick 13

A Anhang **V**
A.1 Java-Code V

Literaturverzeichnis **XVI**

Abbildungsverzeichnis

3.1 Klassendiagramm 5
3.2 Schematische Übersicht der GUI 10

Die Abbildungen in der Dokumentation wurden vom Autor erstellt.
Die Bilder der Karten in der Anwendung wurden mit Ressourcen von Freepik.com erstellt.

Quellcodeverzeichnis

A.1 Klasse `Spiel` V
A.2 Klasse `Karte` VI
A.3 Klasse `Kartenstapel` VII
A.4 Klasse `Spieler` X
A.5 Klasse `SpielerMensch` XI
A.6 Klasse `SpielerPC` XII

1 Einleitung

1.1 Problemstellung und Ziel der Arbeit

Ziel der Arbeit ist es, die Entwicklung einer funktionsfähigen Anwendung zu dokumentieren. Dabei soll der Ablauf der Softwareentwicklung reflektiert und ein Überblick über das entwickelte Programm gegeben werden. Um die Softwareentwicklung praktisch und konkret an einem Beispiel umzusetzen, wird das Spiel „Mau-Mau“ als Anwendung entwickelt. Die Problematik besteht darin, dieses Spiel mit Klassen in der Programmiersprache Java abzubilden, welche für einen übersichtlichen Code geeignet sind und zudem die Objekte des realen Spiels sinnvoll abbilden.

1.2 Aufbau der Arbeit

Anschließend an die Einleitung werden in Kapitel 2.1 die Grundlagen der Softwareentwicklung dargestellt. Um die Vorgehensweise in dieser Arbeit nachvollziehen zu können, werden dazu die Phasen der Softwareentwicklung kurz erläutert. Da für das Klassendiagramm die Modellierungssprache UML verwendet wird, werden in Kapitel 2.2 kurz die Hintergründe zu UML dargestellt. Des Weiteren soll in Kapitel 2.3 die angewandte Programmiersprache Java vorgestellt werden. Um die Softwareentwicklung übersichtlich darzustellen, ist der Hauptteil dieser Arbeit entsprechend den Phasen der Softwareentwicklung gegliedert.[1] Der Hauptteil beginnt in Kapitel 3.1 mit einer Anforderungsanalyse, um festzustellen, welche Probleme die Software lösen muss und welchen Umfang sie haben soll. Konkret werden dazu die Spielregeln geklärt und Rahmenbedingungen, also u.a. Anzahl der Spieler[2] und der Karten, festgesetzt. In der Entwurfsphase (Kapitel 3.2) wird die Software mittels eines Klassendiagramms geplant. Dabei werden die geplanten Funktionalitäten erfasst. Die Implementierung (Kapitel 3.3 und Kapitel 3.4) realisiert den eigentlichen Programmcode. Dabei werden Schlüsselfunktionalitäten erklärt und dementsprechend wird auf einige Methoden näher eingegangen. In Kapitel 3.5 werden die Phasen nach der Implementierung behandelt.

[1] Vgl. Ernst u. a. 2020, 694 ff.

[2] Aufgrund der vielfachen Verwendung von geschlechtsspezifischen Wörtern in dieser Arbeit und der Verwendung dieser Wörter im Code wird auf das generische Maskulinum zurückgegriffen, da der Lesefluss ansonsten zu erschwert wäre.

2 Grundlagen

2.1 Phasen der Softwareentwicklung

Bei der Softwareentwicklung werden sechs Phasen im Software-Lebenszyklus unterschieden. Die Softwareentwicklung in dieser Arbeit soll auf diesen Phasen aufbauen und nacheinander behandelt werden. In der ersten Phase werden Anforderungen an die Software analysiert und in Spezifikationen detailliert beschrieben. In der Entwurfsphase wird die Software modellhaft geplant, wobei vor allem erfasst wird, welche Funktionalitäten die Software beinhalten soll. Dies kann z.B. mit einem Klassendiagramm realisiert werden. Während der Phase der Implementierung wird der Quellcode der Software programmiert. Dabei werden die Strukturen des Entwurfs mittels einer Programmiersprache wie Java umgesetzt. In der Testphase muss die Software vor allem jede der Anforderungen erfüllen. Um mögliche Fehleingaben oder seltene Konstellationen zu prüfen, kann erfahrungsbasiert vorgegangen werden, wobei Fehler selten komplett ausgeschlossen werden können. Falls durch die Software Schäden angerichtet werden können (beispielsweise bei finanziellen Transaktionen), müssen die Tests nach festgelegten Regeln ablaufen. Nach der Testphase kann die Software installiert und genutzt werden. In der Phase nach der Inbetriebnahme geht es um Wartung und Weiterentwicklung. Unter Wartung wird dabei nicht nur Fehlerbehebung verstanden, sondern auch Anpassungen an neue Hardware oder Verbesserungen am Code und an der Performance. Weiterentwicklung bedeutet dagegen, dass neue Funktionalitäten hinzukommen können, da beispielsweise neue Bedürfnisse an die Software entstehen oder Optimierungsmöglichkeiten auffallen.[3]

2.2 Die objektorientierte Programmiersprache Java

Java zählt zu den höheren Programmiersprachen und ist objektorientiert. Objektorientierung bedeutet, dass sich in der Programmierung die Realität abstrakt widerspiegelt. Ausgehend von real existierenden Dingen werden in der Programmierung die entsprechenden Klassen als Grundgerüst

[3] Vgl. Ernst u. a. 2020, 694 ff.

erstellt und davon ausgehend Objekte erzeugt. Mit Java lässt sich das konsequent und vergleichsweise einfach umsetzen. Dies wird in Kapitel 3 praktisch umgesetzt. Historisch betrachtet entwickelte sich Java nachfolgend zu den Programmiersprachen Simula, Smalltalk und C++. Mit der Sprache C++ erlangte die Objektorientierung weite Verbreitung. Java baute auf den Erfahrungen und Konzepten von C++ auf, sodass sich die Syntax sehr ähnelt. Es fielen programmiertechnisch fehleranfällige Funktionen weg, um anwenderfreundlicher zu werden. Spätestens ab Mitte der 90er Jahre verbreitete sich Java sprunghaft schnell und ist bis heute eine der populärsten Programmiersprachen.[4]

Als Grundlage für diese Arbeit und die Programmierung der Anwendung wurden die Bücher ‚Grundkurs Java' und ‚Java als erste Programmiersprache' herangezogen.[5]

2.3 UML (Unified Modeling Language)

Für die Entwurfsphase der Software wird in dieser Arbeit ein Klassendiagramm in der Modellierungssprache UML erstellt. Die Entwicklung von UML ist eng mit der Objektorientierung verbunden, da mit ihr komplexere Anforderungen an die Entwurfsphase der Softwareentwicklung gestellt waren bzw. Software auf einen höheren Abstraktionsgrad gehoben wird. Im gleichen Jahrzehnt, in dem Java sprunghaft an Verbreitung gewann, begann sich auch UML als standardisierte Modellierungssprache durchzusetzen.[6] Das Klassendiagramm in dieser Arbeit wird entsprechend der Modellierungssprache UML dargestellt.[7]

[4] Vgl. Abts 2020, S. 1; vgl. Ernst u. a. 2020, S. 415–416.
[5] Vgl. Abts 2020; vgl. Goll und Heinisch 2016.
[6] Vgl. Booch u. a. 2006, S. 18.
[7] Vgl. van Randen u. a. 2016, S. 5–17.

3 Softwareentwicklung eines Spiels mittels Java

3.1 Anforderungen an das Spiel und die Spielregeln

Von Mau-Mau gibt es unterschiedliche Spielformen und -regeln. In dieser Arbeit wird sich ansatzweise auf das „Spielregelbüchlein aus Altenburg“ bezogen.[8] Um die Anwendung noch in einem möglichst übersichtlichen Rahmen zu halten, wurden die Regeln hier etwas modifiziert bzw. teilweise weggelassen. Auf die Möglichkeit, weitere Spielregeln zu implementieren, wird kurz in Kapitel 3.5.2 eingegangen. Grundlegend spielt der Anwender gegen den Computer, daher gibt es zwei Spieler. Es wird mit einem französischen Skatblatt von 32 Karten gespielt. Von einem Ziehstapel werden die Karten gezogen und auf einen Ablagestapel werden sie offen abgelegt.

Spielbeginn: Zu Beginn ziehen beide Spieler fünf Karten vom Ablagestapel. Es wird eine erste Karte vom Ziehstapel gezogen und offen auf den Ablagestapel gelegt.

Spielablauf: Die Spieler sind abwechselnd am Zug. Während des Spielzuges darf man eine Karte von der Hand ablegen, vorausgesetzt die Karte hat die gleiche Farbe oder den gleichen Wert wie die oberste Karte vom Ablagestapel. Ist das nicht möglich, muss eine Karte gezogen werden. Danach endet der Zug. Legt der Gegner eine Sieben, muss entweder eine Strafkarte gezogen oder mit einer Sieben gekontert werden. Je nachdem wie oft gekontert wurde, müssen dementsprechend Karten von dem Spieler gezogen werden, der nicht mehr kontern kann, maximal also vier. Wird wegen einer Sieben gezogen, geht danach der normale Zug weiter. Es darf nicht mehr gekontert werden.

Spielende: Ziel des Spiels ist es, alle Handkarten auf den Ablagestapel abzulegen oder so lange durchzuhalten, bis der Gegner seine zehnte Handkarte zieht. Schafft es ein Spieler, alle Karten von der Hand abzulegen, so gewinnt derjenige. Zieht ein Spieler die zehnte Handkarte, so verliert derjenige.

[8] Vgl. Altenburger Spielkartenfabrik 1981, 127 ff.

3.2 Klassendiagramm als Entwurf

Folgendes Klassendiagramm wird entwickelt und in diesem Kapitel erklärt (siehe Abbildung 3.1):

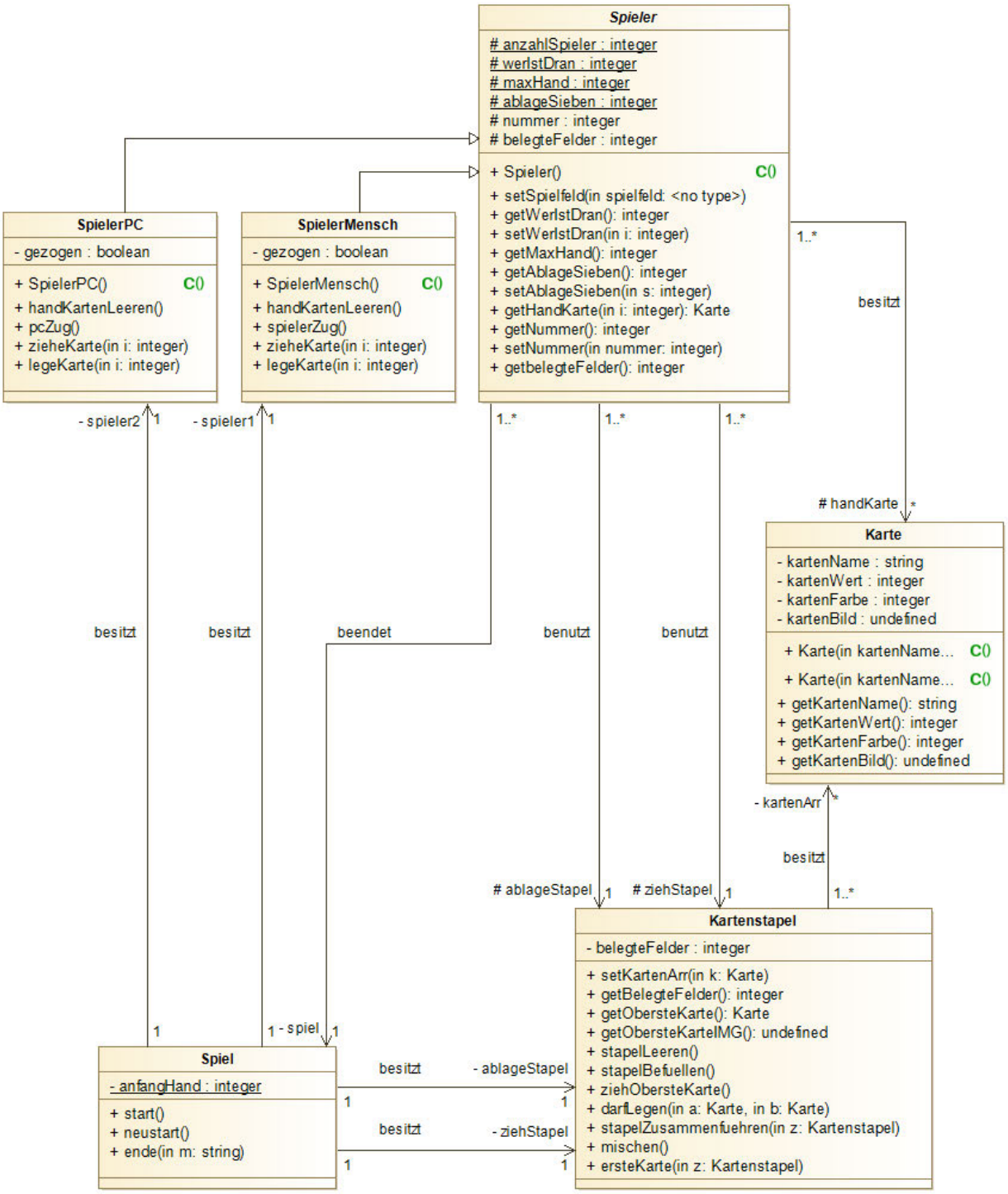

Abbildung 3.1: Klassendiagramm

Nachdem die Anforderungen an die Software erfasst wurden, können damit die Klassen mit den nötigen Funktionalitäten identifiziert werden. Da in der Software ein Kartenspiel nachgebildet wird, welches auch real von mehreren Menschen gespielt werden kann, können hier die Klassen dem realen Spiel nachempfunden werden. Somit ergeben sich Spieler als Klassen. In dieser Arbeit wird ein Spieler als Anwender vorgesehen, welcher gegen den Computer spielt. Insofern werden zwei verschiedene Klassen `SpielerMensch` und `SpielerPC` vorgesehen, die sich in ihren Methoden teils unterscheiden. Der Mensch wird im Gegensatz zum Computer mit der grafischen Oberfläche (GUI, Graphical User Interface) interagieren. Da sie sich aber in den meisten Attributen und Methoden gleichen, wird eine generalisierte Klasse `Spieler` vorgesehen. Die Spieler interagieren mit der Klasse `Kartenstapel`, von deren Objekten Karten gezogen oder auf denen Karten abgelegt werden können. Dabei müssen die Kartenstapel weitere Funktionen ermöglichen: Es dürfen nur unter bestimmten Bedingungen Karten abgelegt werden und unter bestimmten Bedingungen müssen die Stapel gemischt oder umgestapelt werden. Die Karte selbst wird auch als eine Klasse betrachtet. Sie ist vor allem Datenträger der verschiedenen Kartenwerte. Sowohl die Kartenstapel als auch die Spieler können Karten als Objekte aufnehmen. Das Spiel an sich wird ebenfalls als Klasse betrachtet. Es erzeugt die Objekte aller anderen Klassen und stellt die Main-Funktion, also den eigentlichen Programmaufruf zur Verfügung. Zudem stellt sie die grundlegenden Funktionen von Start, Neustart und Spielende zur Verfügung. Im Klassendiagramm wird die GUI nicht berücksichtigt, da es darin maßgeblich darum gehen soll, die Objekte der realen Welt zu erfassen.[9] Die GUI erhält möglichst keine eigenen Attribute und Methoden der eigentlichen Spiellogik. Je nachdem wie die Schnittstelle zum Anwender realisiert werden soll, rufen die entsprechenden Buttons die Methoden der anderen Klassen auf.

3.3 Implementierung der Spiellogik

3.3.1 Klassen und Vererbung

Die Klassen werden in Java folgendermaßen implementiert: Für jede Klasse wird eine eigene Datei angelegt, wobei der Klassenname mit dem Dateinamen übereinstimmt. In der Klasse werden ihre Attribute und Methoden definiert. Folgendes Vorgehen wird für alle Klassen und Methoden

[9] Vgl. Goll und Heinisch 2016, S. 61.

angewendet:

Implementierung der Klasse:

```
public class Kartenstapel { Inhalt der Klasse }
```

Implementierung eines Attributs innerhalb der Klasse:

```
private int belegteFelder = 0;
```

Implementierung der Methoden innerhalb der Klasse:

```
public void stapelLeeren() { Inhalt der Methode }
```

Die Klasse `Spieler` wird eine Superklasse, die ihre Attribute und Methoden an die Klassen `SpielerMensch` und `SpielerPC` vererbt. Dazu ist in diesen Unterklassen der Zusatz `extends Spieler` nötig (siehe Listing A.5 Z. 1). Des Weiteren muss die besondere Übernahme des Konstruktors beachtet werden. Der Konstruktor ist in der Klasse `Spieler` vollständig implementiert. In der Unterklasse wird er folgendermaßen übernommen (siehe Listing A.5 Z. 5-8). Von diesen Klassen können dann je nach Bedarf Objekte erzeugt werden. Das passiert bei dieser Anwendung in der Klasse `Spiel`, welche auch die Funktion `main()` enthält. Über verschiedene Methoden werden diese Objekte dann miteinander in Beziehung treten.

3.3.2 Assoziationen

Eine Herausforderung in der objektorientierten Programmierung stellt die Realisierung der Beziehungen, insbesondere der Abhängigkeiten zwischen den Klassen, dar. Damit zwei Objekte miteinander in Beziehung treten können, müssen sie voneinander Kenntnis haben. Dies muss mindestens in eine Richtung realisiert werden. In dieser Arbeit wurde den Klassen, die mit anderen interagieren sollen, die jeweils andere Klasse als Attribut zugewiesen, entweder direkt über den Konstruktor (siehe z.B. Listing A.1 Z. 5) oder über eine `set`-Methode (siehe z.B. Listing A.1 Z. 22).[10]

3.3.3 Karten und Kartenstapel

Grundlegende Elemente des Spiels sind die Spielkarten und die Kartenstapel. Das wird mit den Klassen `Karte` und `Kartenstapel` realisiert. Die Klasse `Karte` dient dazu, die Daten der Karten (Name, Wert, Farbe und Bild) zu speichern und über `get`-Methoden zur Verfügung zu

[10] Vgl. Lahres u. a. 2021, 120 ff.

stellen. Die Kartenstapel haben als maßgebliches Attribut ein Array vom Datentyp der Klasse `Karte` und können somit mehrere Karten aufnehmen (siehe Listing A.3 Z. 4). Alternativ könnten hierfür Arraylisten zum Einsatz kommen. In dieser Arbeit werden trotzdem einfache Arrays genutzt, um die grundlegenden Kontrollstrukturen von Java zu veranschaulichen und einen Einstieg in die Programmiersprache zu gewährleisten. Die Klasse `Kartenstapel` verfügt über mehrere Methoden, die die Bearbeitung der Stapel ermöglichen, also sie zu befüllen, zu leeren, Karten zu ziehen oder zu legen, Kartenstapel umzustapeln und die Karten zu mischen. Des Weiteren wird die Abfrage zur Verfügung gestellt, ob eine Karte entsprechend der Spielregeln abgelegt werden darf.

3.3.4 Mischen-Algorithmus

Da das Mischen einen besonderen Algorithmus darstellt, soll auf diese Methode näher eingegangen werden (siehe Listing A.3 Z. 86-96). Hierfür wird der Fisher-Yates-Algorithmus genutzt. Bei diesem Algorithmus wird beginnend beim ersten Element der Reihe nach jedes Element mit einem zufällig ausgewähltem anderen Element ausgetauscht. Das Element, welches an der Reihe ist, wird nur mit einem Element getauscht, welches einen höheren oder den gleichen Index besitzt. Der Aufwand steigt bei diesem Algorithmus je nach Feldgröße linear, da jedes Element einmalig bearbeitet wird. Somit beträgt die Laufzeit O(n). Bei diesem Algorithmus ist sichergestellt, dass alle Elemente nach dem Mischen auch noch die gleichen wie vorher, also keine doppelt vorhanden sind oder gelöscht wurden. Das wird dadurch sichergestellt, dass immer zwei Elemente gegenseitig den Platz tauschen. Dadurch, dass einmal das ganze Feld durchlaufen wird, ist sichergestellt, dass jedes Element einen zufälligen Platz erhält.[11]

3.3.5 Spielzug

Der Spielzug wird als eine Methode realisiert, welche vom jeweiligem Actionlistener (siehe Kapitel 3.4) aufgerufen wird und ihrerseits weitere Methoden aufruft. Da die Objektorientierung von Problemen der realen Welt ausgeht, orientieren sich auch die Methoden an den Abläufen des realen Spiels. Im Spielverlauf interagieren Spieler, Karten und Kartenstapel über Methodenaufrufe miteinander und beeinflussen somit ihre jeweiligen Attribute. Die Spielzüge zwischen Mensch und Computer unterscheiden sich in einigen Punkten, haben aber grundlegend einen ähnlichen

[11] Vgl. Sedgewick und Wayne 2017, S. 97; vgl. Black 2023.

Ablauf. Die Unterscheidung liegt vor allem in der Auswahl der Karte. Während der Mensch einen Button wählen kann und dieser Button die Handkarte vorgibt, werden beim Computer mittels `for`-Schleife alle Handkarten nacheinander abgefragt. Über die Methode `spielerZug()` wird die Methode `zieheKarte()` aufgerufen falls der Kartenplatz leer ist, ansonsten wird die Methode `legeKarte()` aufgerufen (siehe Listing A.5 Z. 20-30). Beim Computer wird dagegen mittels `for`-Schleife solange versucht eine Karte abzulegen, bis das erste leere Element im Feld erreicht wird, dann wird eine Karte gezogen (siehe Listing A.6 Z. 29-42). Im Falle einer Sieben sind zusätzliche Abfragen implementiert, die diese `for`-Schleife ggf. erneut durchlaufen lassen, da der Zug in dem Fall noch nicht beendet ist.

Da die beiden Methoden `zieheKarte()` und `legeKarte()` wesentlicher Bestandteil der Spiellogik sind, soll auf diese Methoden näher eingegangen werden. Zwischen `SpielerMensch` und `SpielerPC` unterscheiden sich diese Methoden nur im Umgang mit der GUI, daher wird im Folgenden nur `SpielerMensch` behandelt. Durch das Integer-Attribut `i` wird den Methoden zugewiesen, auf welche Karte im Feld sie angewendet werden sollen. Beim Ziehen wird dem aktuellen Element im Feld `handKarte[]` eine Karte zugewiesen und das Attribut `belegteFelder` erhöht (siehe Listing A.5 Z. 43-45). Diese Zuweisung erfolgt durch einen Methodenaufruf von `ziehStapel`. Von dort wird die oberste Karte gezogen, mittels `return` wird sie übergeben und vom Stapel gelöscht (siehe Listing A.3 Z. 46-52). Das Feld `handKarte[]` ist dabei vom Typ `Karte` und zeigt dabei auf Objekte dieser Klasse. Des Weiteren sind in der Methode `if`-Abfragen enthalten, welche sich auf den Umgang mit Strafkarten beziehen (siehe Listing A.5 Z. 34-41). Es wechselt erst der Spieler, wenn alle Strafkarten gezogen wurden und der normale Zug abgeschlossen wurde. Falls der Stapel beim Ziehen leer wird, wird die Methode `stapelZusammenfuehren()` aufgerufen (siehe Listing A.3 Z. 67-84). Wie im echten Spiel verbleibt die oberste Karte des Ablagestapels und die restlichen Karten werden gemischt und auf den Ziehstapel gelegt. Wird beim Ziehen die zehnte Karte gezogen, wird die Methode `ende()` aufgerufen, da dann verloren wurde und ein Neustart durchgeführt wird. Die Methode `legeKarte()` funktioniert nach einem ähnlichem Prinzip. Umgekehrt wird hier dem Ablagestapel über die Methode `setKartenArr()` die aktuelle Handkarte zugewiesen und von der Hand gelöscht (siehe Listing A.5 Z. 58-65). Bevor abgelegt werden kann, wird über eine `if`-Abfrage geprüft, ob der Spieler ablegen darf (siehe Listing A.5 Z. 37). Dazu wird die Methode `darfLegen()` aufgerufen, welche die Handkarte mit der obersten Karte vom Ablagestapel vergleicht (siehe Listing A.3 Z. 54-65). Bei der Sieben ist an der Stelle

die Besonderheit, dass keine Sieben mehr abgelegt bzw. gekontert werden darf, falls schon die erste Strafkarte gezogen wurde. Das wird mittels Attribut `gezogen` abgefragt. Umgekehrt darf abgelegt werden, wenn eine Sieben liegt und darauf eine Sieben gekontert werden kann.[12] Wird eine Sieben abgelegt, wird die Variable `ablageSieben` hochgezählt (siehe Listing A.5 Z. 57). Über diese Variable kann abgefragt werden, wie viele Strafkarten ggf. gezogen werden müssen. Nach jedem Legen wird geprüft, ob die letzte Handkarte abgelegt wurde. Sollte das der Fall sein, wird die Methode `ende()` aufgerufen.

3.4 Implementierung der GUI (Graphical User Interface)

Folgendes Schema vermittelt einen Überblick über die Komponenten der GUI, auf die in diesem Kapitel näher eingegangen wird (siehe Abbildung 3.2):

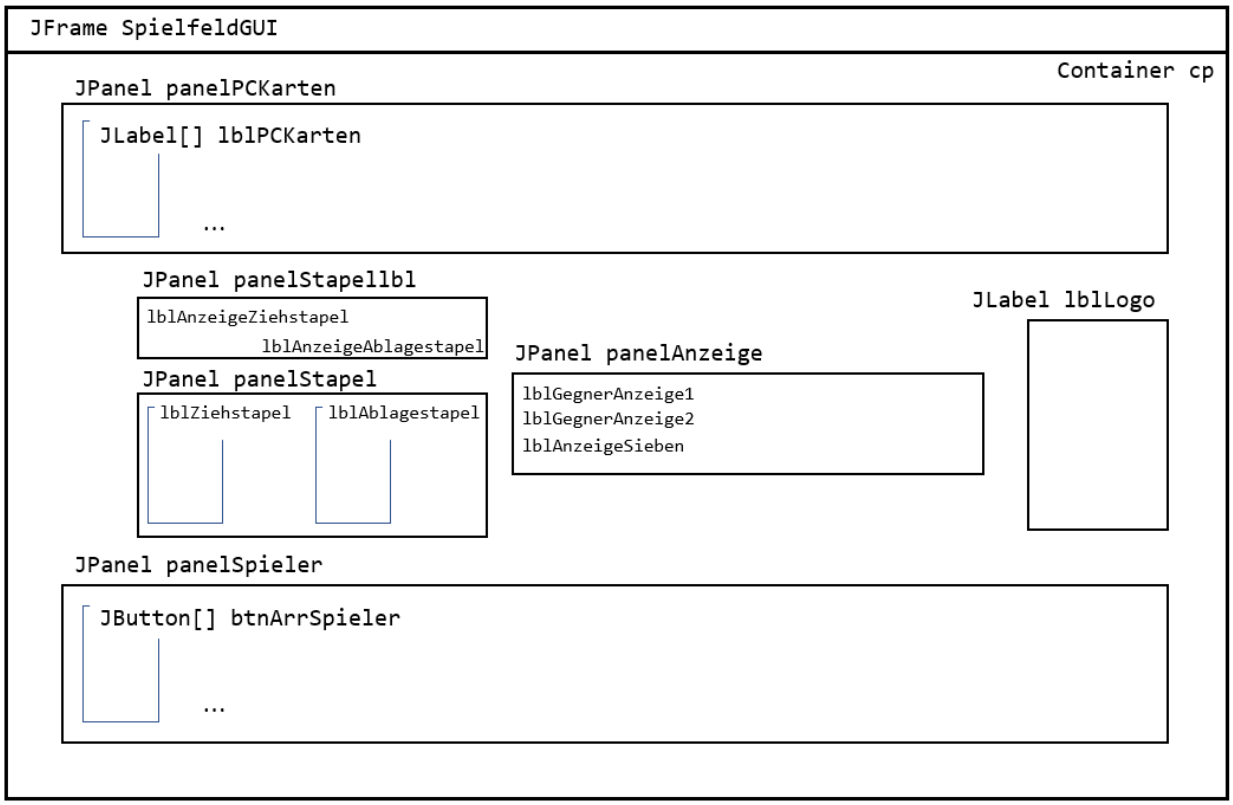

Abbildung 3.2: Schematische Übersicht der GUI

[12] Zu beachten ist hierbei, dass die Karte Sieben im Code den Kartenwert `1` besitzt.

Um eine GUI für eine Java-Anwendung zu entwickeln, bietet sich die Möglichkeit an, Tools als Hilfsmittel zu nutzen, die automatisch den Code generieren. Für diese Arbeit diente zu Beginn der integrierte Designer des Java-Editors von Gerhard Röhner als Hilfsmittel.[13] Der automatisch generierte Code wurde dann manuell angepasst um anhand selbst vergebener Bezeichnungen nachvollziehen zu können, welches Element für welche Aufgaben verantwortlich ist. Bei der Implementierung der GUI werden die Anforderungen an das Spiel grafisch als Schnittstelle für den Anwender umgesetzt. Für die GUI werden in dieser Arbeit die Swing-Klassen verwendet. Die nachfolgend genannten Klassen mit vorangestellten „J“ im Namen sind jeweils Swing-Klassen.[14]

Der Aufbau wird folgendermaßen realisiert: Beim Aufruf des Programms öffnet sich ein Fenster, welches in Java mittels `JFrame` realisiert wird. In der Anwendung wird dazu die Klasse `SpielfeldGUI` eine Unterklasse von `JFrame`. Um Komponenten in der GUI platzieren und positionieren zu können, wird die gesamte Fläche des Fensters zu einem Container deklariert. Der Container stellt selbst eine Komponente dar, auf der andere Komponenten platziert werden können. Die Felder der Klasse `JPanel` sind ebenfalls Container und unterteilen die Oberfläche in verschiedene Bereiche. Die beiden Klassen, die den Spielablauf visualisieren, sind `JLabel` und `JButton`. `JLabel` dient ausschließlich der Anzeige und kann in der Anwendung sowohl Texte als auch Bilder anzeigen. Die Klasse `JButton` wird für die Spielerkarten verwendet und dient dazu, per Mausklick Karten zu ziehen oder zu legen. Anhand des Schemas wird ersichtlich, wie die Kompenenten innerhalb des `JFrame` angeordnet werden.

Damit die anderen Klassen mit der GUI interagieren können, werden Actionlistener benötigt. Ein Actionlistener ist ein Ereignisempfänger und kann bei einem empfangenen Event einen Code aufrufen lassen. Die Ereignisempfänger müssen bei der Ereignisquelle registriert werden. In der Beispielanwendung sind die Actionlistener anonym implementiert.[15] Es können zwei Ereignisse stattfinden: Ein Buttonklick löst einen Spielerzug aus und startet einen Timer, welcher selbst wieder ein Ereignis darstellt. Mit Ablauf des Timers wird der Zug des Computers ausgelöst. Die Actionlistener lösen beim jeweiligen Ereignis die Spielzug-Methoden aus und aktualisieren die Anzeigen auf der Oberfläche. Der Actionlistener stellt somit die Schnittstelle zwischen GUI und eigentlicher Spiellogik dar.

[13] Röhner o. D.
[14] Vgl. Abts 2020, S. 377.
[15] Vgl. Goll und Heinisch 2016, S. 647.

3.5 Phasen nach der Implementierung

3.5.1 Testphase

Beim Testen wurden vor allem alle Anforderungen geprüft, welche aus den Spielregeln hervorgehen. Um die einzelnen Methoden in ihren Funktionen zu testen, wurde teils mit Testklassen gearbeitet und teils mit unsortierten oder vorsortierten Karten, um bestimmte Konstellationen zu prüfen. Beispielsweise musste geprüft werden, ob der Gegner wirklich richtig reagiert, wenn er eine Sieben zieht, danach eine Strafkarte zieht und dann selbst nochmal ablegen darf. Da in dieser Anwendung auf eine freie Eingabe von Zahlen oder Textzeichen verzichtet wurde, sind die möglichen Fehleingaben beschränkt. Aber auch bei einer solchen kleinen Anwendung können Fehler auftreten, die schwer vorherzusehen sind und mit mehrmaligem Testen entdeckt werden müssen, ggf. auch erst in der späteren Phase der Wartung.

3.5.2 Inbetriebnahme, Wartung und Weiterentwicklung

Die Inbetriebnahme bei dieser Anwendung wird mit Hilfe der IDE Eclipse ausgeführt. Eclipse bietet die Möglichkeit, den Quellcode in eine ausführbare jar-Datei zu exportieren. Eine solche Datei kann dem Anwender zur Verfügung gestellt werden und lässt sich direkt ausführen. Die Wartung betrifft vor allem die Behebung der Fehler, die während der Ausführung beim Anwender noch entdeckt werden könnten. Für die Wartung würde sich aber ebenfalls anbieten, den Code noch weiter zu optimieren. Beispielsweise könnten die Arrays in Arraylisten umgewandelt und dadurch der Code vereinfacht werden. Auch wäre zu diskutieren, ob die Klassen und ihre Methoden weiter zu optimieren wären. So könnten die Spieler-Klassen durch ein Interface noch übersichtlicher dargestellt werden und bestimmte Code-Anteile in die generalisierte Klasse verlagert werden. Die Software bietet viele Möglichkeiten weiterentwickelt zu werden. Mehrere Spieler, mehr Sonderregeln oder speicherbare Spielstände wären naheliegende Beispiele. Bei der Implementierung wurde darauf geachtet, hier schon möglichst offen für Weiterentwicklungen zu programmieren. Beispielsweise ist die aktuelle Zahl der Handkarten als eine Variable definiert, die ohne größere Umstände verändert werden könnte. Auch die Objektorientierung selbst bietet hier die Möglichkeit der Erweiterung, da z.B. ein Spieler als Objekt mehrfach erzeugt werden könnte. Anpassungen an der grafischen Oberfläche wären trotzdem noch nötig.

4 Fazit

4.1 Zusammenfassung

Die Dokumentation der Software dieser Arbeit wurde anhand der Phasen der Softwareentwicklung nachgezeichnet. Beginnend bei der Anforderungsanalyse wurden maßgeblich die Spielregeln geklärt und somit die Grundlage dafür gelegt, die Objekte des realen Spiels zu identifizieren und daraus ein Klassendiagramm abzuleiten. Das Klassendiagramm legt das Grundgerüst für die Software und gibt einen Überblick, welche Funktionalitäten implementiert werden müssen. Indem die Implementierung der wichtigsten Methoden der Spiellogik erklärt wurde, konnte die Vorgehensweise trotz des relativ umfangreichen Codes verdeutlicht werden. Zudem konnte gezeigt werden, wie sich Objekte der realen Welt (z.B. Karten und Spieler) vom Entwurf bis zur Implementierung in Java widerspiegeln lassen. In den Phasen nach der Implementierung wurde dargestellt, wie die Software in Betrieb genommen wird und welche Weiterentwicklungsmöglichkeiten es danach noch geben kann.

4.2 Kritische Würdigung und Ausblick

Der Einstieg in die Programmiersprache lässt sich vor allem mit Vorkenntnissen in der Objektorientierung gut bewerkstelligen. Es zeigen sich allerdings schon zu Beginn vielfältige Möglichkeiten, wie sich Probleme verschieden lösen und bestimmte Vorgehensweisen immer weiter optimieren lassen. Spätestens während der Implementierung haben sich immer wieder Möglichkeiten gefunden, den Code besser und übersichtlicher darzustellen oder zu strukturieren. Insofern stellt diese Arbeit nicht die optimalste Lösung der gestellten Anforderungen zur Verfügung, sondern kann als anschaulicher Einstieg in die objektorientierte Programmierung mit Java betrachtet werden. Verschiedene Schwierigkeiten der Objektorientierung, z.B. die Assoziationen, stellen Herausforderungen dar, die in den Lehrbüchern zwar mit Beispielen dargestellt sind, aber erst in der praktischen Umsetzung ganzer Projekte erprobt und erlernt werden können. Insofern kann diese Arbeit einen Beitrag dazu leisten.

A Anhang

A.1 Java-Code

Die Kommentare wurden für die Darstellung in der Dokumentation entfernt, um den Code etwas kürzer und übersichtlicher darzustellen. In den Original-Dateien sind Kommentare zur kurzen Erklärung einzelner Funktionen vorhanden. Der Übersicht halber wurde darauf verzichtet, die Klasse `SpielfeldGUI` im Anhang darzustellen, da sie sehr viel Raum einnimmt und in der Dokumentation nur oberflächlich auf die GUI eingegangen wird.

```
1  public class Spiel
2  {
3      private Kartenstapel ziehStapel = new Kartenstapel();
4      private Kartenstapel ablageStapel = new Kartenstapel();
5      private SpielerMensch spieler1 = new SpielerMensch(1,
       ablageStapel, ziehStapel, this);
6      private SpielerPC spieler2 = new SpielerPC(2, ablageStapel,
       ziehStapel, this);
7      private SpielfeldGUI spielfeld = new SpielfeldGUI(spieler1,
       spieler2, ablageStapel, ziehStapel);
8
9      private static int anfangHand = 5;
10
11     public static void main(String[] args)
12     {
13         Spiel spiel = new Spiel();
14         spiel.start();
15     }
16
17     public void start()
18     {
19         spielfeld.setVisible(true);
20         ziehStapel.setSpielfeld(spielfeld);
21         ablageStapel.setSpielfeld(spielfeld);
22         spieler1.setSpielfeld(spielfeld);
23         spieler2.setSpielfeld(spielfeld);
24
25         neustart();
26     }
27
28     public void neustart()
29     {
```

```
30          spieler.setAblageSieben(0);
31          spieler1.handKartenLeeren();
32          spieler2.handKartenLeeren();
33          ablageStapel.stapelLeeren();
34          ziehStapel.stapelBefuellen();
35
36          for (int i = 0; i < anfangHand; i++)
37          {
38              spieler1.zieheKarte(i);
39          }
40
41          for (int i = 0; i < anfangHand; i++)
42          {
43              spieler2.zieheKarte(i);
44          }
45
46          ablageStapel.ersteKarte(ziehStapel);
47
48          spielfeld.anzeigenAktualisieren();
49          spielfeld.setGegnerAnzeige1 ("");
50          spielfeld.setGegnerAnzeige2 ("");
51
52          Spieler.setWerIstDran(1);
53      }
54
55      public void ende (String m)
56      {
57          neustart();
58          spielfeld.spielendeDlg(m);
59      }
60  }
```

Listing A.1: Klasse `Spiel`

```
1  public class Karte
2  {
3     private String kartenName;
4     private int kartenWert;
5     private int kartenFarbe;
6     private ImageIcon kartenBild;
7
8     public Karte (String kartenName, int kartenWert, int
      kartenFarbe, String kartenBild)
9     {
10       this.kartenName = kartenName;
11       this.kartenWert = kartenWert;
12       this.kartenFarbe = kartenFarbe;
```

```
13        this.kartenBild = new ImageIcon(getClass().getResource(
    kartenBild));
14        this.kartenBild.setImage(this.kartenBild.getImage().
    getScaledInstance(140,212,Image.SCALE_DEFAULT));
15     }
16
17     public Karte (String kartenName, int kartenWert, int
    kartenFarbe)
18     {
19        this.kartenName = kartenName;
20        this.kartenWert = kartenWert;
21        this.kartenFarbe = kartenFarbe;
22     }
23
24     public String getKartenName() {return kartenName;}
25
26     public int getKartenWert() {return kartenWert;}
27
28     public int getKartenFarbe() {return kartenFarbe;}
29
30     public ImageIcon getKartenBild() {return kartenBild;}
31  }
```

Listing A.2: Klasse `Karte`

```
1  public class Kartenstapel
2  {
3      private SpielfeldGUI spielfeld;
4      private Karte[] kartenArr = new Karte [32];
5      private int belegteFelder = 0;
6
7      public void setSpielfeld(SpielfeldGUI spielfeld)
8      {
9          this.spielfeld = spielfeld;
10     }
11
12     public void setKartenArr (Karte k) {kartenArr[belegteFelder
    ++] = k;}
13
14     public int getBelegteFelder() {return belegteFelder;}
15
16     public Karte getObersteKarte()
17     {
18         if (0 == belegteFelder) return null;
19         else return kartenArr[belegteFelder - 1];
20     }
21
```

```
22     public ImageIcon getObersteKarteIMG() {return kartenArr[
   belegteFelder - 1].getKartenBild();}
23
24     public void stapelLeeren()
25     {
26         for (int i = 0; i < 32; i++)
27         {
28             kartenArr[i] = null;
29         }
30         belegteFelder = 0;
31     }
32
33     public void stapelBefuellen()
34     {
35         kartenArr[0] = new Karte ("Herz␣7", 1, 1, "/img/herz_7.
   jpg");
36         kartenArr[1] = new Karte ("Herz␣8", 2, 1, "/img/herz_8.
   jpg");
37         .
38         .
39         .
40         //Es werden alle 32 Karten erzeugt...
41         belegteFelder = 32;
42
43         mischen();
44     }
45
46     public Karte ziehOberstesKarte()
47     {
48         Karte hilfsKarte = kartenArr[belegteFelder - 1];
49         kartenArr[belegteFelder - 1] = null;
50         belegteFelder--;
51         return hilfsKarte;
52     }
53
54     public boolean darfLegen(Karte a, Karte b)
55     {
56         if (a.getKartenFarbe() == b.getKartenFarbe())
57         {
58             return true;
59         }
60         else if (a.getKartenWert() == b.getKartenWert())
61         {
62             return true;
63         }
64         else return false;
```

```
65      }
66
67      public void stapelZusammenfuehren (Kartenstapel z)
68      {
69          for (int i = 0; i < (this.belegteFelder - 1); i++)
70          {
71              z.kartenArr[i] = this.kartenArr[i];
72              this.kartenArr[i] = null;
73          }
74
75          z.belegteFelder = (this.belegteFelder - 1);
76
77          this.kartenArr[0] = this.kartenArr[this.belegteFelder -
    1];
78          this.kartenArr[this.belegteFelder - 1] = null;
79          this.belegteFelder = 1;
80
81          z.mischen();
82
83          spielfeld.setLblAblageStapel(getObersteKarteIMG());
84      }
85
86      public void mischen()
87      {
88          for (int i = 0; i < (belegteFelder); i++)
89          {
90              int r = i + (int) (Math.random() * (belegteFelder - i
    ));
91
92              Karte hilf = kartenArr[i];
93              kartenArr[i] = kartenArr[r];
94              kartenArr[r] = hilf;
95          }
96      }
97
98      public void ersteKarte (Kartenstapel z)
99      {
100         kartenArr[belegteFelder++] = z.ziehObersteKarte();
101         if (1 == kartenArr[0].getKartenWert()) {Spieler.
    setAblageSieben(1);}
102         spielfeld.setLblAblageStapel(getObersteKarteIMG());
103
104         spielfeld.anzeigenAktualisieren();
105     }
106 }
```

Listing A.3: Klasse `Kartenstapel`

```
1  public class Spieler
2  {
3     protected Kartenstapel ablageStapel;
4     protected Kartenstapel ziehStapel;
5     protected SpielfeldGUI spielfeld;
6     protected Spiel spiel;
7     protected static int anzahlSpieler = 2;
8     protected static int werIstDran = 1;
9     protected static int maxHand = 10;
10    protected static int ablageSieben = 0;
11
12    protected Karte[] handKarte = new Karte [maxHand];
13    protected int nummer;
14    protected int belegteFelder = 0;
15
16    public Spieler(int nummer, Kartenstapel ablageStapel,
      Kartenstapel ziehStapel, Spiel spiel)
17    {
18       this.nummer = nummer;
19       this.ablageStapel = ablageStapel;
20       this.ziehStapel = ziehStapel;
21       this.spiel = spiel;
22    }
23
24    public void setSpielfeld(SpielfeldGUI spielfeld) {this.
      spielfeld = spielfeld;}
25
26    public static int getWerIstDran() {return werIstDran;}
27
28    public static void setWerIstDran()
29    {
30       if (anzahlSpieler == werIstDran) { werIstDran = 1;}
31       else werIstDran++;
32    }
33
34    public static void setWerIstDran(int i) {werIstDran = i;}
35
36    public static int getMaxHand() {return maxHand;}
37
38    public static int getAblageSieben() {return ablageSieben;}
39
40    public static void setAblageSieben(int s) {ablageSieben = s;}
41
42    public Karte getHandKarte(int i) {return handKarte[i];}
43
44    public int getNummer(){return nummer;}
```

```
45
46     public void setNummer(int nummer) {this.nummer = nummer;}
47
48     public int getBelegteFelder() {return belegteFelder;}
49 }
```

Listing A.4: Klasse `Spieler`

```
1  public class SpielerMensch extends Spieler
2  {
3      private boolean gezogen = false;
4
5      public SpielerMensch (int nummer, Kartenstapel ablageStapel,
       Kartenstapel ziehStapel, Spiel spiel)
6      {
7          super (nummer, ablageStapel, ziehStapel, spiel);
8      }
9
10     public void handKartenLeeren()
11     {
12         for (int i = 0; i < maxHand; i++)
13         {
14             handKarte[i] = null;
15             spielfeld.setBtnArrSpieler(i, null);
16         }
17         belegteFelder = 0;
18     }
19
20     public void spielerZug(int i)
21     {
22         if (nummer == werIstDran)
23         {
24             if (null == getHandKarte(i)) zieheKarte(i);
25             else legeKarte(i);
26
27             spielfeld.setGegnerAnzeige1 ("");
28             spielfeld.setGegnerAnzeige2 ("");
29         }
30     }
31
32     public void zieheKarte (int i)
33     {
34         if (0 < ablageSieben)
35         {
36             gezogen = true;
37             ablageSieben--;
38         }
```

```
39          if (false == gezogen) setWerIstDran();
40
41          if (0 == ablageSieben) gezogen = false;
42
43          handKarte[i] = ziehStapel.ziehOberstekarte();
44          spielfeld.setBtnArrSpieler(i, handKarte[i]);
45          belegteFelder++;
46
47          if (null == ziehStapel.getOberstekarte()) ablageStapel.
    stapelZusammenfuehren(ziehStapel);
48
49          if (0 == belegteFelder) {spiel.ende("Gewonnen");}
50          else if (maxHand == belegteFelder) {spiel.ende("Verloren"
    );}
51      }
52
53      public void legeKarte(int i)
54      {
55          if (((true == ablageStapel.darfLegen(handKarte[i],
    ablageStapel.getOberstekarte())) && (0 == ablageSieben)) ||
    ((0 < ablageSieben) && ((false == gezogen) && (1 == handKarte[
    i].getKartenWert()))))
56          {
57              if (1 == handKarte[i].getKartenWert()) {ablageSieben
    ++;}
58              Karte hilfsKarte = handKarte[i];
59              handKarte[i] = null;
60              belegteFelder--;
61
62              ablageStapel.setKartenArr(hilfsKarte);
63              spielfeld.setLblAblageStapel(ablageStapel.
    getOberstekarteIMG());
64              spielfeld.setBtnArrSpieler(i, null);
65              setWerIstDran();
66
67              if (0 == belegteFelder) {spiel.ende("Gewonnen");}
68              else if (maxHand == belegteFelder) {spiel.ende("
    Verloren");}
69          }
70      }
71  }
```

Listing A.5: Klasse `SpielerMensch`

```
1  public class SpielerPC extends Spieler
2  {
3      private boolean gezogen = false;
```

```
4
5     public SpielerPC (int nummer, Kartenstapel ablageStapel,
  Kartenstapel ziehStapel, Spiel spiel)
6     {
7         super (nummer, ablageStapel, ziehStapel, spiel);
8     }
9
10    public void handKartenLeeren()
11    {
12        for (int i = 0; i < maxHand; i++)
13        {
14            handKarte[i] = null;
15            spielfeld.setLblPCKarten(i, "leer");
16        }
17        belegteFelder = 0;
18    }
19
20    public void pcZug ()
21    {
22        if (nummer == werIstDran)
23        {
24            int hilfZiehen = 0;
25            int hilfAblegen = 0;
26
27            while (nummer == werIstDran)
28            {
29                for (int i = 0; ((i < (belegteFelder + 1)) && (
  nummer == werIstDran)); i++)
30                {
31                    if (null == getHandKarte(i))
32                    {
33                        zieheKarte(i);
34                        hilfZiehen++;
35                        break;
36                    }
37                    else if (((true == ablageStapel.darfLegen(
  handKarte[i], ablageStapel.getObersteKarte())) && (0 ==
  ablageSieben)) || ((0 < ablageSieben) && ((false == gezogen)
  && (1 == handKarte[i].getKartenWert()))))
38                    {
39                        legeKarte(i);
40                        hilfAblegen++;
41                    }
42                }
43            }
```

```
44              spielfeld.setGegnerAnzeige1 ("Zug␣vom␣Gegner:␣"  +
    hilfZiehen + "␣Karte(n)␣gezogen");
45              spielfeld.setGegnerAnzeige2 ("und␣"  + hilfAblegen +
    "␣Karte(n)␣gelegt.");
46          }
47      }
48
49      public void zieheKarte (int i)
50      {
51          if (0 < ablageSieben)
52          {
53              gezogen = true;
54              ablageSieben--;
55          }
56          if (false == gezogen) setWerIstDran();
57
58          if (0 == ablageSieben) gezogen = false;
59
60          handKarte[i] = ziehStapel.ziehOberstekarte();
61          spielfeld.setLblPCKarten(i, "");
62          belegteFelder++;
63
64          if (null == ziehStapel.getOberstekarte()) ablageStapel.
    stapelZusammenfuehren(ziehStapel);
65
66          if (0 == belegteFelder) {spiel.ende("Verloren");}
67          else if (maxHand == belegteFelder) {spiel.ende("Gewonnen"
    );}
68      }
69
70      public void legeKarte(int i)
71      {
72          if (1 == handKarte[i].getKartenWert()) {ablageSieben++;}
73          Karte hilfsKarte = handKarte[i];
74          handKarte[i] = null;
75          belegteFelder--;
76
77          ablageStapel.setKartenArr(hilfsKarte);
78          spielfeld.setLblAblageStapel(ablageStapel.
    getOberstekarteIMG());
79          setWerIstDran();
80
81          handKarte[i] = handKarte[belegteFelder];
82          handKarte[belegteFelder] = null;
83          spielfeld.setLblPCKarten(belegteFelder, "leer");
84
```

```
85          if (0 == belegteFelder) {spiel.ende("Verloren");}
86          else if (maxHand == belegteFelder) {spiel.ende("Gewonnen"
    );}
87      }
88  }
```

Listing A.6: Klasse `SpielerPC`

Literaturverzeichnis

ABTS, Dietmar (2020). *Grundkurs JAVA*. 11. Aufl. Wiesbaden: Springer Vieweg. ISBN: 978-3-658-30494-2.

ALTENBURGER SPIELKARTENFABRIK (1981). *Erweitertes Spielregelbüchlein aus Altenburg*. 3. Aufl. Dresden: Altenburger Spielkartenfabrik.

BLACK, Paul E. (2023). *"Fisher-Yates shuffle", in Dictionary of Algorithms and Data Structures [online]*. (abgerufen am 04.07.2023). URL: *https://www.nist.gov/dads/HTML/fisherYatesShuffle.html*.

BOOCH, Grady u. a. (2006). *Das UML Benutzerhandbuch*. 1. Aufl. München: Addison-Wesley Verlag. ISBN: 978-3-8273-2295-1.

ERNST, Hartmut u. a. (2020). *Grundkurs Informatik*. 7. Aufl. Wiesbaden: Springer Vieweg. ISBN: 978-3-658-30331-0.

GOLL, Joachim und HEINISCH, Cornelia (2016). *Java als erste Programmiersprache*. 8. Aufl. Wiesbaden: Springer Vieweg. ISBN: 978-3-658-12118-1.

LAHRES, Berhard u. a. (2021). *Objektorientierte Programmierung*. 5. Aufl. Bonn: Rheinwerk Verlag. ISBN: 978-3-8362-8317-5.

RÖHNER, Gerhard (o. D.). *Java-Editor*. (abgerufen am 04.07.2023). URL: *https://javaeditor.org/*.

SEDGEWICK, Robert und WAYNE, Kevin (2017). *Computer Science*. 1. Aufl. Princeton: Addison-Wesley. ISBN: 978-0-13-407642-3.

VAN RANDEN, Hendrik Jan u. a. (2016). *Einführung in UML*. 1. Aufl. Wiesbaden: Springer Vieweg. ISBN: 978-3-658-14412-8.